



Bitirme Projesi Raporu

MAT4400

Parçacık Sürü Optimizasyonunun İncelenmesi ve Uygulamaları

Ümit Anıl Kılıç
20050541

Doç.Dr. Arzu ÜNAL

Fen Fakültesi
Matematik
Haziran, 2024

Etik

Ankara Üniversitesi Fen Fakültesi Matematik Bölümü Bitirme Projesi yazım kurallarına uygun bir şekilde hazırladığım bu proje raporunda yer alan bilgilerin bilimsel olduğunu, bilgilerin düzenlenmesi ve üretilmesi aşamalarında Etik ve Ahlak kurallarına uygun davrandığımı ve kullandığım bütün referanslara Kaynaklar Bölümü'nde yer verdiğimi kabul ve beyan ederim.

Ümit Anıl KILIÇ

Özet

Parçacık Sürü Optimizasyonu, kuş sürüleri veya balıkların sosyal davranışından esinlenen evrimsel bir hesaplama tekniğidir. Optimize edilen problem hakkında çok az varsayımda bulunduğu veya hiç varsayımda bulunmadığı ve aday çözümlerin çok geniş alanlarını arayabildiği için meta-sezgisel yöntemler sınıfına dahil edilir. Bu projede, optimizasyonun önemli bir alt sınıfı olan Parçacık Sürü Optimizasyonu (PSO) yöntemi tanıtılacak, yöntemle ilişkin algoritma ve akış şeması verilecek ve PSO'nun çeşitli uygulama alanları hakkında bilgi verilecektir. PSO'nun temel prensiplerinin yanı sıra, PSO'nun nasıl çalıştığı, temel bileşenleri ve optimizasyon sürecindeki adımlar çeşitli örnekler üzerinde ele alınacaktır.

İçindekiler

Etik	1
Özet	2
1 Giriş	4
2 Parçacık Sürü Optimizasyonu (PSO)	5
2.1 Optimizasyon Nedir?	5
2.1.1 Deterministik Yaklaşımlar	5
2.1.2 Stokastik Temelli Teknikler	6
2.1.3 Stokastik Öğrenme Yaklaşımları	6
2.1.4 Sezgisel Yöntemler	6
2.1.5 Metasezgisel Yöntemler (Metaheuristics)	6
2.2 Parçacık Sürü Optimizasyonu Nedir?	7
2.2.1 PSO'nun Kullanım Alanları	8
2.2.2 PSO Çeşitleri	8
2.3 Parçacık Sürü Optimizasyonu'nun Çalışma Metodolojisi	10
2.3.1 Atalet Ağırlığı Stratejileri	12
2.3.2 PSO Algoritması	13
2.3.3 PSO Akış Diyagramı	14
3 PSO Uygulamaları	15
3.1 Problem-1	15
3.1.1 Farklı Atalet Ağırlığı Stratejilerinin Karşılaştırılması	19
3.2 Problem-2	23
3.3 Problem-3	28
4 Sonuç	29
Kaynakça	30

Bölüm 1

Giriş

Bu çalışmada, Parçacık Sürü Optimizasyonu (PSO) algoritmasının farklı problemler üzerine uygulamaları incelenmektedir. Optimizasyon, belirli bir amaç doğrultusunda en iyi sonucu elde etmek için parametrelerin ayarlanması sürecidir. Optimizasyon yöntemleri, belirli bir hedef fonksiyonun maksimum veya minimum değerini bulmayı amaçlar. Bu yöntemler, gradyan temelli, evrimsel, meta-sezgisel ve rassal yöntemler gibi alt sınıflara ayrılır.

Gradyan temelli yöntemler, hedef fonksiyonun gradyanını kullanarak çözüme iteratif olarak yaklaşırken, evrimsel algoritmalar doğadan ilham alarak, popülasyon tabanlı arama stratejileri kullanır. Meta-sezgisel yöntemler ise, belirli bir probleme özgü olmayan ve geniş bir uygulama alanına sahip genel yaklaşımlardır. Rassal yöntemler, çözümleri rastgele seçerek çözüm uzayını keşfetmeyi amaçlar.

PSO, doğada sürüler halinde hareket eden kuş ve balık gibi canlıların kolektif davranışlarını taklit ederek, optimizasyon problemlerine çözümler üreten etkili bir meta-sezgisel yöntemdir. Evrimsel algoritmalar, genetik algoritmalar ve diferansiyel evrim gibi doğadaki evrim sürecinden ilham alır ve popülasyon temelli arama yaparak çözüm bulur.

PSO algoritmasının temel prensibi, rastgele başlatılan parçacıkların, belirli bir amaç fonksiyonunun minimum veya maksimum değerini bulmak için hem kendi en iyi konumlarına (P_{best}) hem de sürünün en iyi konumuna (G_{best}) göre hareket etmesidir. Bu çalışmada, PSO'nun temel bileşenleri, çalışma prensibi ve çeşitli stratejilerle performansının nasıl artırılacağı ele alınmıştır. Ayrıca, PSO'nun farklı uygulama alanlarına dair örnekler verilmiş ve bu bağlamda Drop-Wave fonksiyonu üzerinde yapılan deneyler ile algoritmanın etkinliği değerlendirilmiştir.

Bölüm 2

Parçacık Sürü Optimizasyonu (PSO)

2.1 Optimizasyon Nedir?

Optimizasyon, belirli bir amaç veya hedef doğrultusunda en iyi sonucu elde etmek için bir sistemin parametrelerini ayarlama sürecidir. Başka bir deyişle, optimizasyon, belirli kısıtlar altında bir hedef fonksiyonunu maksimuma çıkarma veya minimuma indirme işlemidir. Optimizasyon, mühendislikten ekonomiye, işletmeden yapay zekaya kadar birçok alanda kritik bir rol oynar.

Optimizasyon problemleri genel olarak üç ana bileşenden oluşur:

1. **Karar Değişkenleri:** Değiştirilebilen ve hedef fonksiyonu etkileyen parametrelerdir.
2. **Hedef Fonksiyonu:** Maksimize edilmek veya minimize edilmek istenen fonksiyondur.
3. **Kısıtlar:** Karar değişkenlerinin uyması gereken koşullardır.

Optimizasyon problemlerinin çözümünde kullanılan yöntemler, problemin yapısına ve özelliklerine bağlı olarak değişir. Optimizasyon yöntemleri genel olarak aşağıdaki kategorilere ayrılır:

2.1.1 Deterministik Yaklaşımlar

Deterministik yaklaşımlar, belirsizlik içermeyen ve her çalıştırıldığında aynı sonuçları üreten yöntemlerdir. Bu kategoride yer alan yöntemler şunlardır:

- **Doğrusal Programlama (Linear Programming):** Doğrusal eşitlikler ve eşitsizliklerle tanımlanan bir sistemde bir hedef fonksiyonunu optimize etmeyi amaçlar.
- **Tamsayılı Programlama (Integer Programming):** Çözüm kümesi, tüm değişkenlerin tamsayı olduğu doğrusal programlama problemidir.
- **Konveks Optimizasyon Yöntemleri (Convex Optimization):** Amaç fonksiyonu ve kısıtların konveks olduğu problemlerin çözümünde kullanılır.
- **Doğrusal Olmayan Programlama (Nonlinear Programming):** Doğrusal olmayan fonksiyonların ve/veya kısıtların bulunduğu optimizasyon problemlerini içerir.

2.1.2 Stokastik Temelli Teknikler

Stokastik temelli teknikler, rastgelelik veya belirsizlik unsurlarını içeren yöntemlerdir. Bu teknikler şunlardır:

- **Gradyan Temelli Yöntemler (Gradient-Based Methods):** Amaç fonksiyonunun gradyanını kullanarak optimum çözümü bulmaya çalışır.
- **Gradyansız Yöntemler (Gradient-Free Methods):** Gradyan bilgisi gerektirmeden optimizasyon yapar, genellikle karmaşık veya gri kutu modellerde kullanılır.

2.1.3 Stokastik Öğrenme Yaklaşımları

Bu yaklaşımlar, veri tabanlı yöntemler kullanarak belirsizlikler altında öğrenmeyi içerir. İki ana kategorisi vardır:

- **Gözetimli Öğrenme Yaklaşımı (Supervised Learning):** Eğitim verisi ve doğru çıktı bilgisi kullanılarak model eğitilir.
- **Gözetimsiz Öğrenme (Unsupervised Learning):** Sadece girdi verisi kullanılarak yapıyı öğrenmeye çalışır, doğru çıkış bilgisi yoktur.

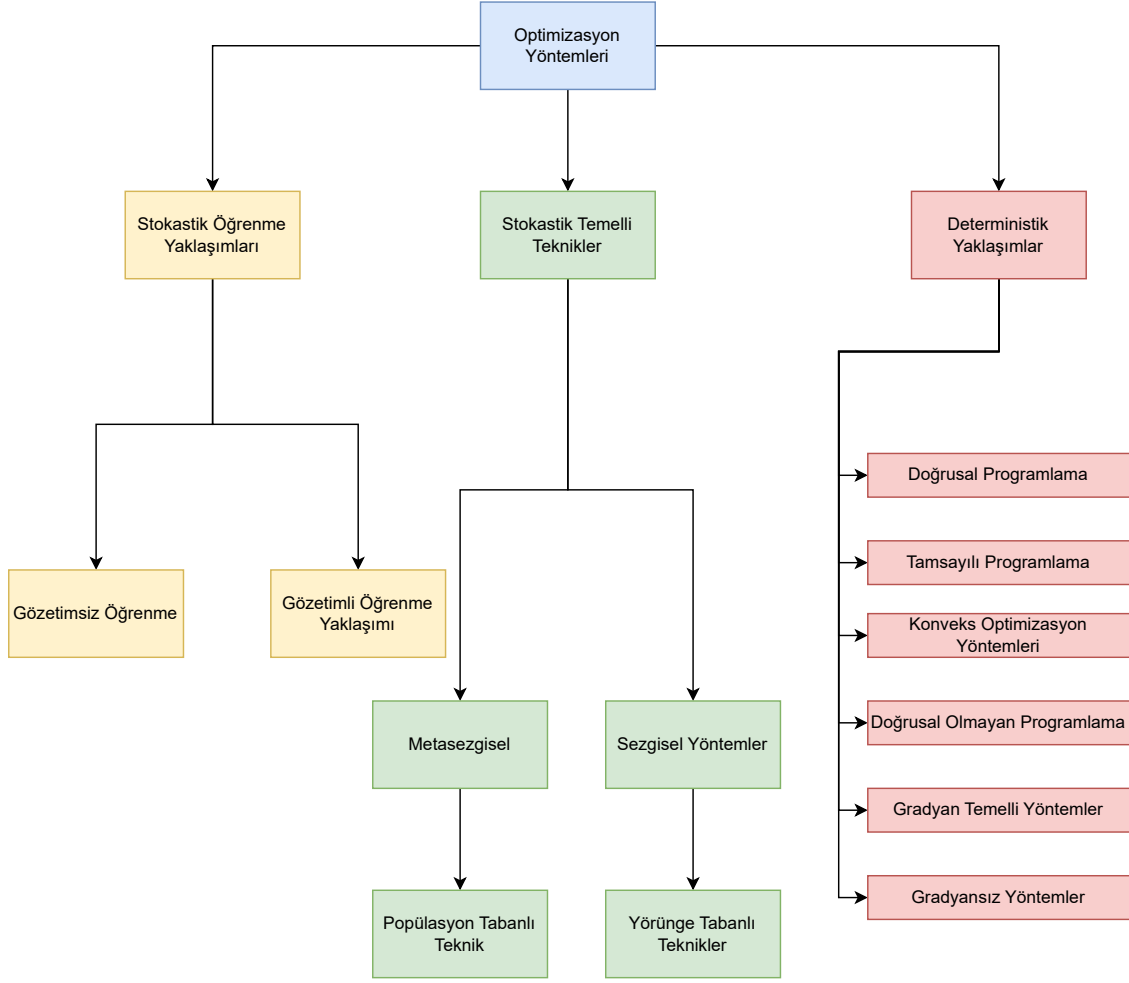
2.1.4 Sezgisel Yöntemler

Sezgisel yöntemler, tam bir çözüm bulmak zor veya imkansız olduğunda kullanılan ve genellikle tatmin edici çözümler sunan tekniklerdir. İki ana kategorisi vardır:

- **Yörünge Tabanlı Teknikler (Trajectory-Based Techniques):** Çözüm uzayında tek bir çözümden başlayarak iyileştirmeler yapar.
- **Popülasyon Tabanlı Teknikler (Population-Based Techniques):** Bir popülasyon (birden çok çözüm) üzerinde çalışarak iyileştirme yapar.

2.1.5 Metasezgisel Yöntemler (Metaheuristics)

Metasezgisel yöntemler, genellikle daha basit sezgisel yöntemlerin bir kombinasyonunu ve bunların ötesinde bir genel arama stratejisini içeren yöntemlerdir. Bu yöntemler, geniş ve karmaşık arama uzaylarında iyi çözümler bulmak için tasarlanmıştır.



Şekil 2.1: Optimizasyon Yöntemlerinin Kategorileri [1]

2.2 Parçacık Sürü Optimizasyonu Nedir?

Parçacık Sürü Optimizasyonu (PSO), kuş ve balık sürülerinin sosyal davranışlarından esinlenerek geliştirilmiş popülasyon tabanlı bir sezgisel algoritmadır. İlk olarak 1995 yılında J. Kennedy ve R.C. Eberhart tarafından ortaya konulmuştur[2]. PSO, optimize edilen problem hakkında çok az varsayımda bulunduğu veya hiç varsayımda bulunmadığı ve aday çözümlerin çok geniş alanlarını arayabildiği için meta-sezgisel bir yöntemdir. Ayrıca PSO, optimize edilen problemin gradyanını kullanmaz; bu da, PSO'nun, en dik iniş, yarı-newton vb. klasik optimizasyon yöntemlerinin gerektirdiği optimizasyon probleminin türevlenebilme şartını gerektirmediği anlamına gelir. Buna karşılık, PSO gibi meta-sezgisel yöntemler en uygun çözümün bulunacağını garanti etmez. PSO, sürekli veya ayrık değerli optimizasyon problemlerini çözmek için kullanılabilir. Diğer taraftan PSO, popülasyon tabanlı yaklaşımı ve aday çözümlerin yinelemeli olarak iyileştirilmesi nedeniyle evrimsel bir algoritma olarak da sınıflandırılabilir. Geleneksel genetik operatörler yerine adaptasyon ve sosyal öğrenme ilkelerini kullanır ve bu da onu evrimsel algoritma ailesinin ayırt edici bir üyesi yapar. Bu benzersiz yaklaşım, PSO'nun evrimsel hesaplamaların temel ilkelerini korurken çok çeşitli optimizasyon problemlerini etkili bir şekilde çözmesine olanak tanır.

2.2.1 PSO'nun Kullanım Alanları

PSO, türevlenemeyen veya doğrusal olmayan, büyük arama uzayı problemlerine uygulanabilen ve iyi bir verimlilikle iyi sonuçlar veren etkili bir optimizasyon yöntemidir[3]. PSO'nun bu esnekliği ve etkinliği, kendisine geniş bir yelpazede kullanım alanı bulma imkânı vermiştir.

PSO, özellikle mühendislik, elektrik mühendisliği, makine öğrenmesi, veri madenciliği, yapay sinir ağları, ekonomi, tıp, robotik ve telekomünikasyon gibi birçok farklı alanda başarıyla uygulanmıştır. Karmaşık ve çok boyutlu optimizasyon problemlerinin çözümünde özellikle etkilidir. [4][5][6][7][8][9]

2.2.2 PSO Çeşitleri

Parçacık Sürü Optimizasyonu (PSO), geniş bir yelpazede optimizasyon problemlerini çözmek için kullanılabilen güçlü bir meta-sezgisel algoritmadır. Temel PSO algoritmasına karşı, yıllar içinde birçok farklı varyasyon geliştirilmiştir. Bu varyasyonlar, özel problemlerin ve ihtiyaçların gereksinimlerini karşılamak için tasarlanmıştır. Aşağıda, en yaygın kullanılan PSO çeşitlerinden bazıları ve temel özellikleri özetlenmiştir:

1. Küresel PSO (G-PSO):

- Temel PSO algoritmasıdır.
- Tüm parçacıklar, en iyi konumu (en iyi çözümü) birbirleriyle paylaşır.
- Parçacıklar, en iyi konumu güncelleyerek ve bu konumu aramak için hareket ederek optimize edilir.

2. Yerel PSO (L-PSO):

- Her parçacık, yalnızca belirli bir komşuluk alanındaki diğer parçacıklarla etkileşime girer.
- Global bilgiyi paylaşmak yerine daha yerel bir arama stratejisi kullanır.
- Hızlı ve verimli bir optimizasyon sağlayabilir, ancak global en iyi çözümü bulma olasılığı daha düşüktür.

3. Uyarlamalı (Adaptif) PSO:

- Parametreler (hız sınırları, ağırlık faktörleri vb.) zamanla veya iterasyonlar boyunca değişebilir.
- Daha esnek ve etkili bir optimizasyon sağlar.
- Farklı problem türlerine ve değişen koşullara uyum sağlayabilir.

4. Kısıtlama Katsayılı PSO:

- Hız sınırları için bir katsayı kullanır ve bu katsayıların optimize edilmesiyle birlikte daha hızlı ve istikrarlı bir yakınsama sağlar.
- Özellikle titreşim problemlerinin önlenmesinde faydalıdır.

5. Çok Amaçlı PSO (MOPSO):

- Birden fazla hedef fonksiyonu olan çok amaçlı optimizasyon problemleri için uyarlanmıştır.
- Parçacıklar, bir dizi hedefi optimize etmeye çalışırken, birbirleriyle rekabet edebilir veya işbirliği yapabilirler.
- Karmaşık ve çok boyutlu problemlerde faydalıdır.

6. İkili PSO:

- Problemin çözümünü ikili (0 ve 1 gibi) bir formatta ifade edilmiş değişkenler üzerinde gerçekleştirir.
- Genellikle genetik algoritmalarla birleştirilerek kullanılır.
- Arttırma/azaltma problemleri ve ayrık (diskret) optimizasyon problemleri için uygundur.

7. Dinamik PSO:

- Çözüm uzayı veya hedef fonksiyon zamanla değişen dinamik optimizasyon problemleri için uygundur.
- Parametrelerin veya stratejilerin değiştirilmesiyle dinamik olarak uyum sağlar.
- Değişen koşullara sahip problemlerde ve gerçek zamanlı optimizasyonda faydalıdır.

2.3 Parçacık Sürü Optimizasyonu'nun Çalışma Metodolojisi

Parçacık Sürü Optimizasyonu (PSO), bir problemin en iyi çözümünü bulmak için birbirleriyle etkileşim halinde olan kuşların (sanal parçacıkların) bir sürüsünü kullanan bir meta-sezgisel algoritmadır. Bu algoritma, kuş sürüsünün yiyecek arama davranışından ilham almıştır.

1. Sürünün Başlatılması: Her parçacık rastgele bir konuma ve hıza atanır. Bu konumlar, problemin çözüm uzayını temsil eder.

2. En İyi Konumların Belirlenmesi: Her parçacık, kendi en iyi çözümünü (kişisel en iyi) ve tüm sürünün en iyi çözümünü (küresel en iyi) takip eder.

3. Hız Güncelleme: Her parçacığın hızı, kişisel en iyi konumuna ve küresel en iyi konuma olan uzaklıklarına göre güncellenir.

4. Konum Güncelleme: Her parçacığın konumu, güncellenmiş hıza göre güncellenir.

5. Tekrarlama: 2. ve 3. adımlar, önceden belirlenmiş bir kriter karşılanana kadar (örneğin, belirli sayıda yineleme veya istenilen doğruluk seviyesi) tekrarlanır.

$$v_i^{t+1} = wv_i^t + c_1r_1(Pbest_i - P_i^t) + c_2r_2(Gbest - P_i^t) \quad (2.1)$$

$$P_i^{t+1} = P_i^t + v_i^{t+1} \quad (2.2)$$

[10]

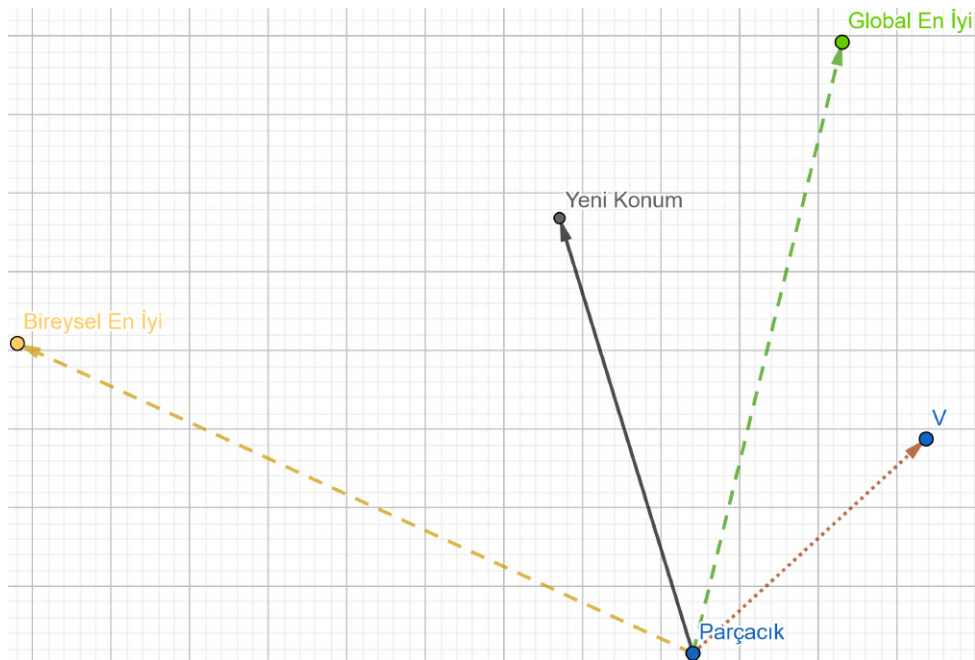
Hız ve konum güncelleme formülleri sırasıyla (2.1) ve (2.2) ile ifade edilir. Bu formüllerde:

- (v_i^t) , parçacık (i) 'nin (t) zamanındaki hızıdır.
- (w) atalet(eylemsizlik, inertia) ağırlığıdır.
- (c_1) ve (c_2) ivme sabitleridir.
- (r_1) ve (r_2) rastgele sayılardır (genellikle 0 ile 1 arasında).
- $(Pbest_i)$, parçacık (i) 'nin kişisel en iyi konumudur.
- $(Gbest)$, küresel en iyi konumdur.
- (P_i^t) , parçacık (i) 'nin (t) zamanındaki konumudur (x_i^t olarak da ifade edilebilir)

Hız güncelleme formülü (2.1), Parçacık Sürü Optimizasyonu (PSO) algoritmasında bir parçacığın hızını güncellemek için kullanılan önemli bir denklemdir. Bu formül, genellikle aşağıdaki 3 ana kısımdan oluşur:

- wv_i^t **[Atalet (Eylemsizlik) Bileşeni]**: Bu bileşen, parçacığın önceki hızını dikkate alarak hareketin bir kısmını korumasına olanak tanır. Parçacığın mevcut hızının bir kısmı, bir sonraki iterasyona aktarılır. Bu, parçacığın hareketine bir süreklilik kazandırır ve ani yön değişikliklerini önlemeye yardımcı olur.
- $c_1r_1(Pbest_i - P_i^t)$ **[Bilişsel (Cognitive) Bileşeni]**: Bu bileşen, parçacığın kendi en iyi pozisyonuna (Pbest) doğru hareket etmesini sağlar. Parçacığın mevcut pozisyonu ile kendi en iyi pozisyonu arasındaki fark dikkate alınarak bir çekim kuvveti uygulanır. Bu, parçacığın kendi deneyimlerinden öğrenmesini ve daha iyi çözümlere yönelmesini sağlar.
- $c_2r_2(Gbest - P_i^t)$ **[Sosyal (Social) Bileşeni]**: Bu bileşen, parçacığın sürünün en iyi pozisyonuna (Gbest) doğru hareket etmesini sağlar. Parçacığın mevcut pozisyonu ile sürünün en iyi pozisyonu arasındaki fark dikkate alınarak bir çekim kuvveti uygulanır. Bu, parçacığın diğer parçacıklardan öğrenmesini ve sürünün genel performansını artırmasını sağlar.

Bu üç bileşenin ağırlıklı toplamı, parçacığın yeni hızını belirler. Bu ağırlıklar, algoritmanın performansını ve yakınsama hızını etkileyen önemli parametrelerdir.



Atalet Ağırlığı (Inertia Weight) Kavramı

Atalet ağırlığı, PSO'nun önemli bir parametresidir ve algoritmanın yakınsama (convergence) ile keşif ve sömürü dengesini (exploration-exploitation trade-off) önemli ölçüde etkiler. Atalet ağırlığı kavramı ilk olarak 1998 yılında Shi ve Eberhart tarafından tanıtılmıştır[11]. Bu kavram, parçacıkların hız güncelleme denkleminde yer alarak küresel ve yerel aramaları daha iyi kontrol etmeyi amaçlar.

- **Keşif (Exploration):** Algoritmanın çözüm uzayında geniş çapta arama yaparak yeni ve çeşitli bölgeleri araştırmasıdır. Bu, global optimumu bulma şansını artırır.
- **Sömürü (Exploitation):** Algoritmanın mevcut iyi çözümler etrafında detaylı arama yaparak daha iyi çözümler bulmaya çalışmasıdır. Bu, yerel optimumları iyileştirmeyi hedefler.

2.3.1 Atalet Ağırlığı Stratejileri

Bansal ve arkadaşları (2011) çalışmasında[3], PSO'da atalet ağırlığını ayarlamak için önerilen çeşitli stratejiler incelenmiştir. Bunlar arasında sabit, rastgele, adaptif, sigmoid artan/azalan, doğrusal azalan, kaotik, kaotik rastgele, osilasyonlu, global-yerel en iyi, benzetimli tavlama, doğal üstel fonksiyon, logaritmik azalan ve eksponansiyel azalan atalet ağırlıkları bulunmaktadır.

Karşılaştırmalı Analiz

Çalışmada 15 farklı atalet ağırlığı stratejisi, beş farklı optimizasyon testi problemi (Sphere, Griewank, Rosenbrock, Rastrigin, Ackley) kullanılarak karşılaştırılmıştır. Deneyler için kullanılan parametreler arasında 50 partiküllü sürü boyutu, 10 karar değişkeni ve maksimum 1000 iterasyon bulunmaktadır. Her strateji 30 simülasyon ile test edilmiştir.

DeneySEL Sonuçlar

Sonuçlar, ortalama hata, ortalama iterasyon sayısı ve elde edilen minimum hata kriterlerine göre değerlendirilmiştir:

- **Kaotik Atalet Ağırlığı Stratejisi:** En yüksek doğruluğa sahip strateji olarak belirlenmiştir.
- **Rastgele Atalet Ağırlığı Stratejisi:** En yüksek verimliliği sunan strateji olarak bulunmuştur.
- **Doğrusal Azalan ve Sabit Atalet Ağırlıkları:** Optimum sonuçlara yakın sonuçlar üreten diğer etkili stratejiler olarak değerlendirilmiştir.

2.3.2 PSO Algoritması

1. Başlat:

- Parçacık sayısı (N) ve boyut (D) belirle.
- Her parçacık için konum (x) ve hız (v) değerlerini rastgele ata.
- Her parçacığın kişisel en iyi konumunu (P_{best}) başlangıç konumu olarak ayarla.
- Sürüdeki global en iyi konumu (G_{best}) bul ve kaydet.

2. Uygunluk Değerlendir:

- Her parçacık için uygunluk (fitness) fonksiyonunu (f) kullanarak uygunluk değerini hesapla.

3. Kişisel ve Global En İyileri Güncelle:

- Her parçacık için:
 - Eğer mevcut uygunluk değeri P_{best} ’ten daha iyiyse, P_{best} ’i güncelle.
 - Eğer herhangi bir parçacığın P_{best} ’i G_{best} ’ten daha iyiyse, G_{best} ’i güncelle.

4. Hız ve Konum Güncelle:

- Her parçacık için:
 - Hız güncelleme formülünü (2.1) kullanarak hızı güncelle.
 - Konum güncelleme formülünü (2.2) kullanarak konumu güncelle.

5. Durdurma Kriteri:

- Maksimum iterasyon sayısına ($maxIter$) ulaşıldıysa veya başka bir durdurma kriteri sağlandıysa dur, aksi takdirde 2. adıma dön.

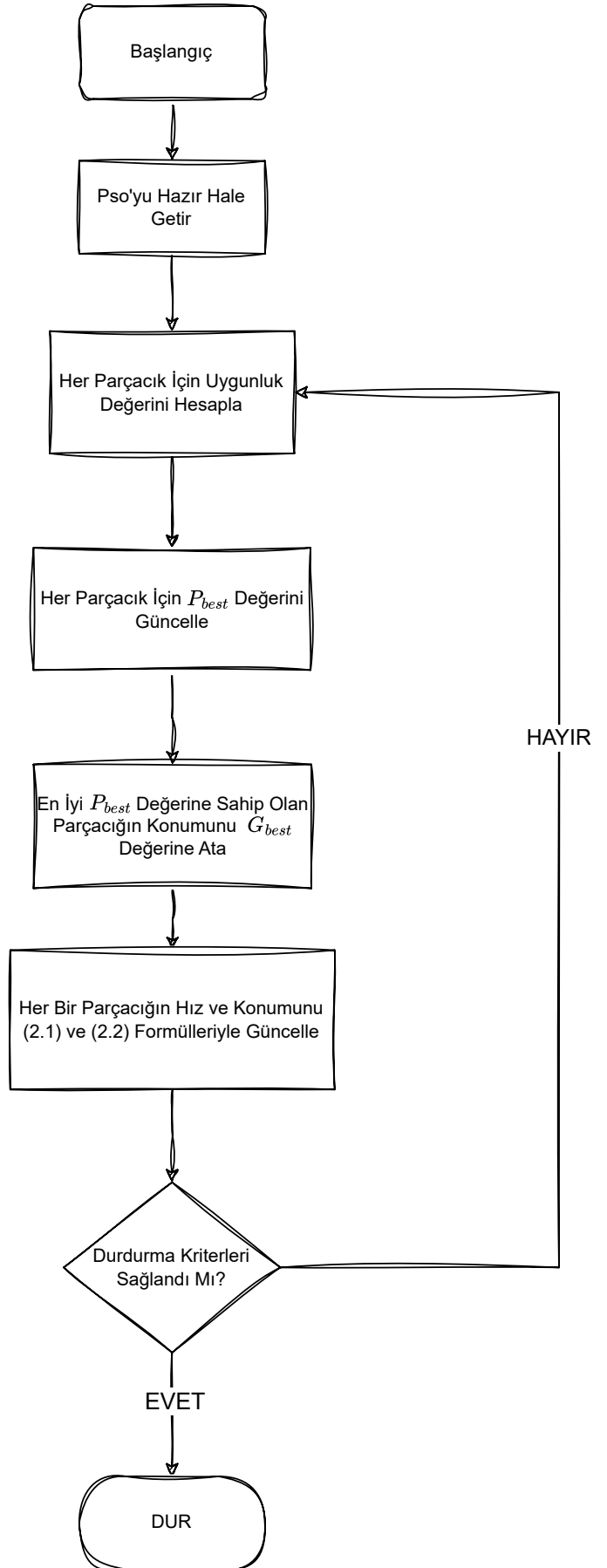
6. Sonuç:

- Global en iyi konum (G_{best}) ve uygunluk değeri ($f(G_{best})$) algoritmanın sonucudur.

2.3.3 PSO Akış Diyagramı

PSO'yu Hazır Hale Getirmek

1. Parçacık sayısı ve boyutu belirlenir.
2. Her parçacığın konum ve hızı rastgele atanır.
3. Her parçacığın en iyi konumu başlangıç konumu olarak ayarlanır.
4. Sürüdeki en iyi global konum bulunur ve kaydedilir.



Bölüm 3

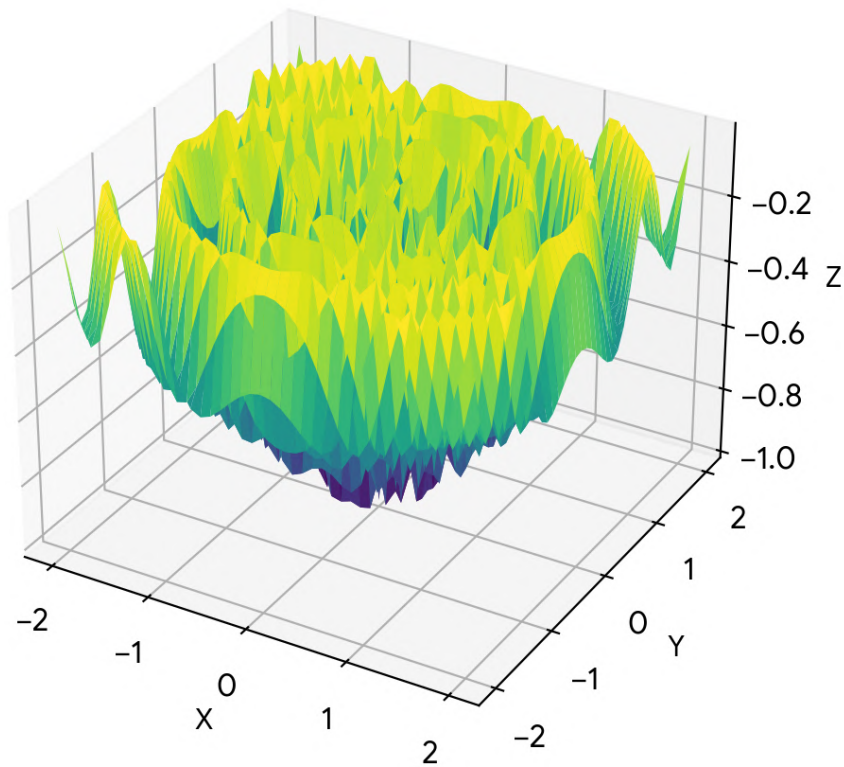
PSO Uygulamaları

3.1 Problem-1

Bu çalışmada, Parçacık Sürüşü Optimizasyonu (PSO) kullanarak Drop-Wave fonksiyonunun ele alacağız. Drop Wave fonksiyonu şu şekilde tanımlanır:

$$f(x, y) = -\frac{1 + \cos(12\sqrt{x^2 + y^2})}{0.5(x^2 + y^2) + 2}$$

Drop-wave Fonksiyonu



Şekil 3.1: Drop-Wave Fonksiyonu

PSO algoritmasını kullanarak bu fonksiyonun minimum noktasını bulmak için, KILIC, U. A. tarafından geliştirilen PSO kütüphanesini kullanacağız[12].

Aşağıdaki adımlar, bu optimizasyon probleminin çözüm sürecini detaylandırmaktadır:

1. Uygunluk (Fitness) Fonksiyonunun Belirlenmesi:

Öncelikle, Drop Wave fonksiyonunu PSO algoritmasında kullanılacak uygunluk fonksiyonu olarak tanımlayalım.

```
// Uygunluk fonksiyonu
dropWaveFunction := func(p pso.Position) float64 {
    // Drop Wave fonksiyonu (https://www.sfu.ca/~ssurjano/drop.html)
    a := 1 + math.Cos(12*math.Sqrt(math.Pow(p.GetCoordinates()[0],2)
        +math.Pow(p.GetCoordinates()[1],2)))
    b := 0.5*(math.Pow(p.GetCoordinates()[0],2)
        +math.Pow(p.GetCoordinates()[1], 2)) + 2
    return -1 * (a / b)
}
```

2. PSO Parametrelerinin Belirlenmesi:

PSO algoritmasının etkin bir şekilde çalışabilmesi için gerekli parametreleri belirlememiz gerekmektedir. Bu parametreler şunlardır:

```
// PSO parametreleri
var inertia float64 = 0.7          // w
var cognitive float64 = 1.5        // c1
var social float64 = 1.5          // c2
var numParticles int = 30          // Parçacık sayısı
var numIterations int = 10         // İterasyon sayısı
var searchSpaceSize float64 = 10   // Arama uzayı boyutu
```

- Parçacık sayısı
- Maksimum iterasyon sayısı
- Öğrenme faktörleri (c_1 ve c_2)
- Atalet ağırlığı (w)

4. Arama Uzayının Kısıtlanması Oluşturulması:

Arama Uzayını $(-10, -10)$ ile $(10, 10)$ arasına kısıtlayacak fonksiyonu tanımlayalım.

```
constraintFunction := func(p *pso.Position) {
    // Sınırların dışına çıkıldığında sınırlara geri dönülür
    if p.GetCoordinates()[0] < -10 {
        p.UpdateCoordinate(-10, p.GetCoordinates()[1])
    } else if p.GetCoordinates()[0] > 10 {
        p.UpdateCoordinate(10, p.GetCoordinates()[1])
    }
    if p.GetCoordinates()[1] < -10 {
        p.UpdateCoordinate(p.GetCoordinates()[0], -10)
    } else if p.GetCoordinates()[1] > 10 {
        p.UpdateCoordinate(p.GetCoordinates()[0], 10)
    }
}
```

4. Parçacıkların Oluşturulması:

Çözüm uzayında rasgele konum ve hızlarla parçacıkları oluşturalım.

```
particles := make([]pso.Particle, 0)
for i := 0; i < numParticles; i++ {
    randomPos := pso.NewPosition(rand.Float64()*searchSpaceSize,
        rand.Float64()*searchSpaceSize) // Rastgele bir konum
    randomVel := pso.NewPosition(rand.Float64()*searchSpaceSize,
        rand.Float64()*searchSpaceSize) // Rastgele bir hız
    particle := pso.NewParticle(i, randomPos, randomVel)
    particles = append(particles, particle) // Parçacık listesine eklenir
}
}
```

5. Sürünün Oluşturulması:

Tüm parçacıkları içeren sürüyü oluşturalım ve başlangıç uygunluk değerlerini hesaplayalım.

```
// Sürü oluşturulur
swarm := pso.NewSwarm(inertia, cognitive, social,
    particles, dropWaveFunction, constraintFunction)
problem1 := pso.NewPSO(&swarm)
problem1.SetIterationCount(numIterations)
```

6. Algoritmanın Çalıştırılması:

Belirlenen iterasyon sayısı boyunca parçacıkların hız ve konum güncellemelerini gerçekleştirerek algoritmayı çalıştıralım. Her iterasyonda en iyi lokal ve global uygunluk değerlerini güncelleyelim.

```
// PSO algoritması çalıştırılır
problem1.Optimize()
bestPosition := swarm.GetGlobalBest()
bestFitness := dropWaveFunction(bestPosition)
fmt.Println("Best Position: ", bestPosition) // En iyi konumu yazdırır
fmt.Println("Best Fitness: ", bestFitness)    // En iyi uygunluk değerini yaz
```

Bu adımları izleyerek, Drop Wave fonksiyonunun minimum noktasını PSO kullanarak etkili bir şekilde bulabiliriz.

Sonuçların Karşılaştırılması

Bu bölümde, PSO algoritması ile elde edilen sonuçları, Drop Wave fonksiyonunun bilinen minimum değeri ile karşılaştıracğız.

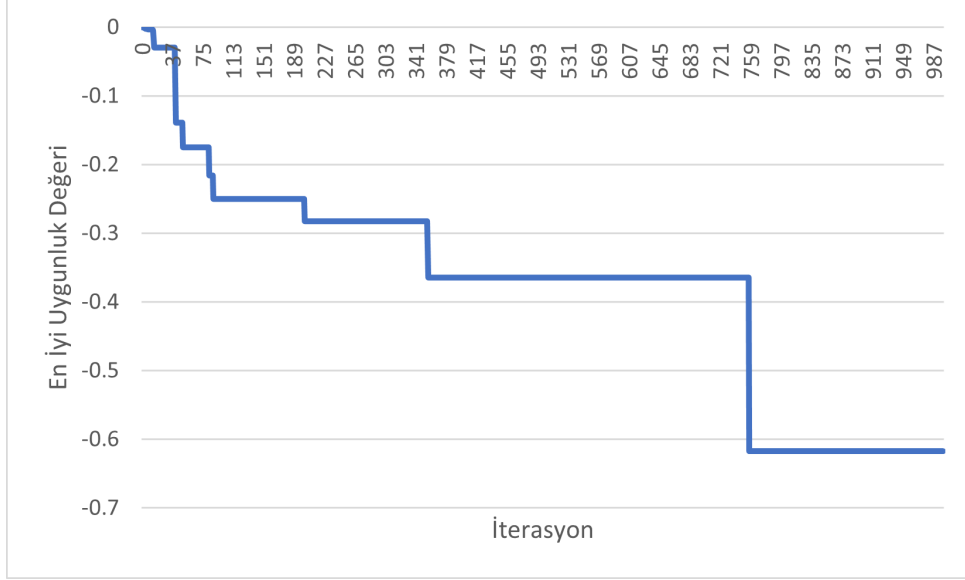
Algoritmanın hesapladığı sonuçlar aşağıdaki gibidir:

- Teorik minimum değer: -1
- Bulunan minimum değer: -0.9349248340111491
- Minimum nokta: $(x, y) = (-0.23283621477862448, 0.45821176910124617)$

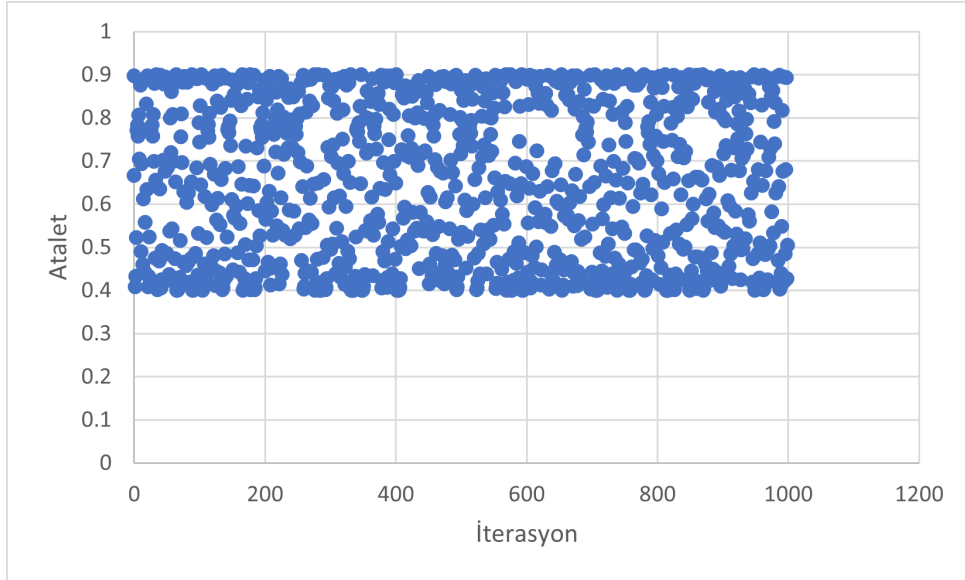
Bu karşılaştırmadan, PSO algoritmasının 10 iterasyonda Drop Wave fonksiyonunun minimum değerine oldukça yakın bir sonuç verdiğini görüyoruz. Algoritmanın bulduğu değer, teorik minimum değerden yaklaşık 0.07 farkla sapmıştır. Bu sapma, PSO algoritmasının doğruluğunu ve fonksiyonun minimum noktasını bulma konusundaki etkinliğini göstermektedir.

3.1.1 Farklı Atalet Ağırlığı Stratejilerinin Karşılaştırılması

Bu kısımda, (2.3.1) bölümünde ele alınan Kaotik, Doğrusal Azalan ve Rastgele Atalet ağırlığı stratejileri, drop-wave fonksiyonu üzerinde uygulanarak algoritmanın performansı değerlendirilmiş arama uzayı $(-1000, -1000)$ ile $(1000, 1000)$ aralığına genişletilmiş ve 1000 iterasyon yapılmıştır.

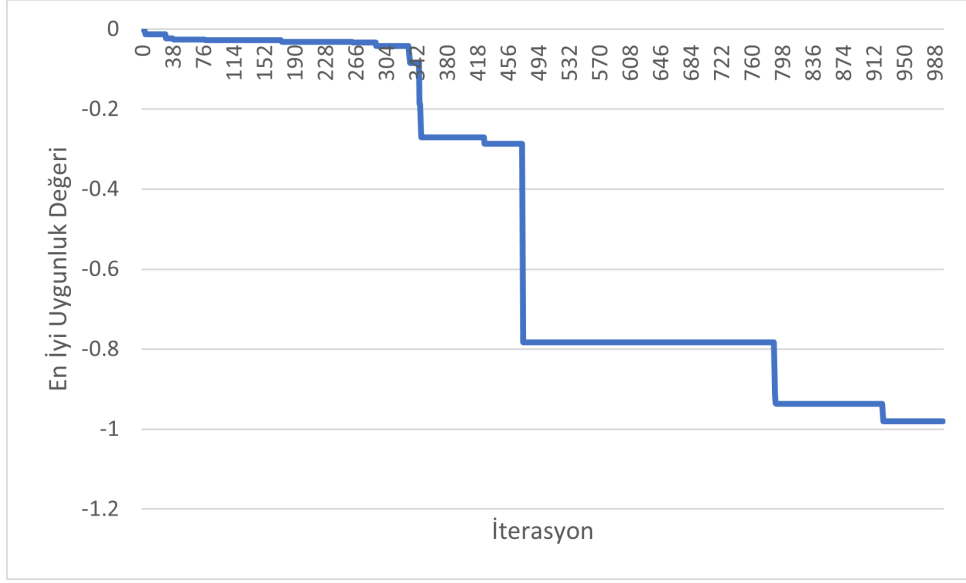


Şekil 3.2: Kaotik Atalet Stratejisi - 1

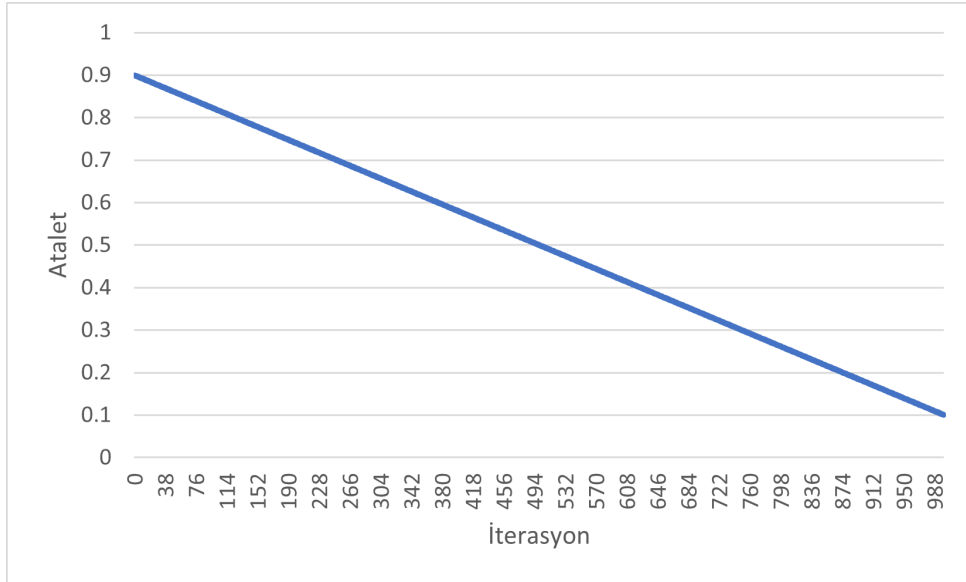


Şekil 3.3: Kaotik Atalet Stratejisi - 2

Kaotik Atalet Stratejisi, atalet değerini her iterasyonda kaotik bir şekilde değiştirmektedir. Bu strateji, arama uzayının rastgele bir şekilde araştırılmasına izin vermektedir ve yerel optimumlardan kaçınmayı amaçlamaktadır. Sonuçlara göre, bu strateji en iyi uygunluk değerine $f(G_{Best}) = -0.61683$, 779 iterasyon sonunda ulaşmıştır

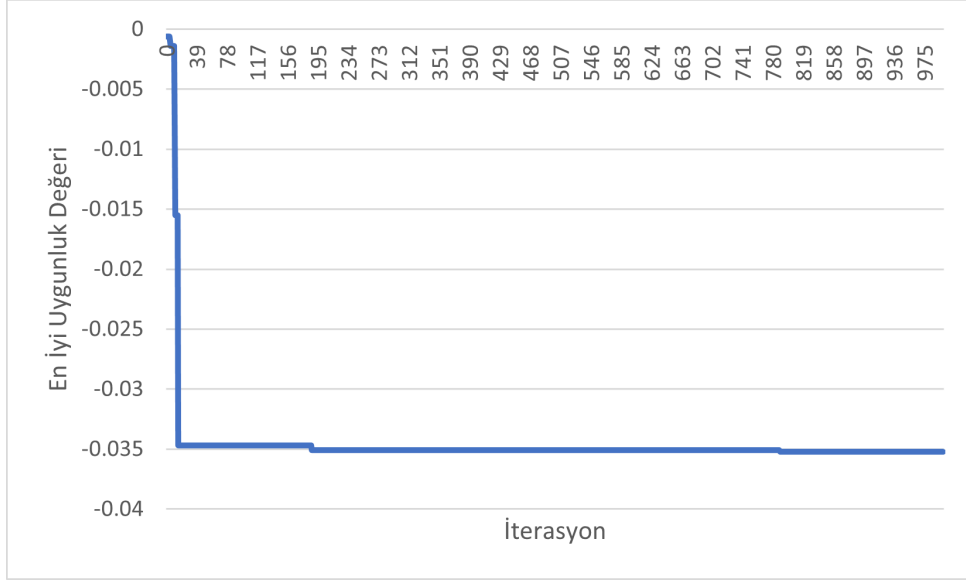


Şekil 3.4: Doğrusal Azalan Atalet Stratejisi - 1

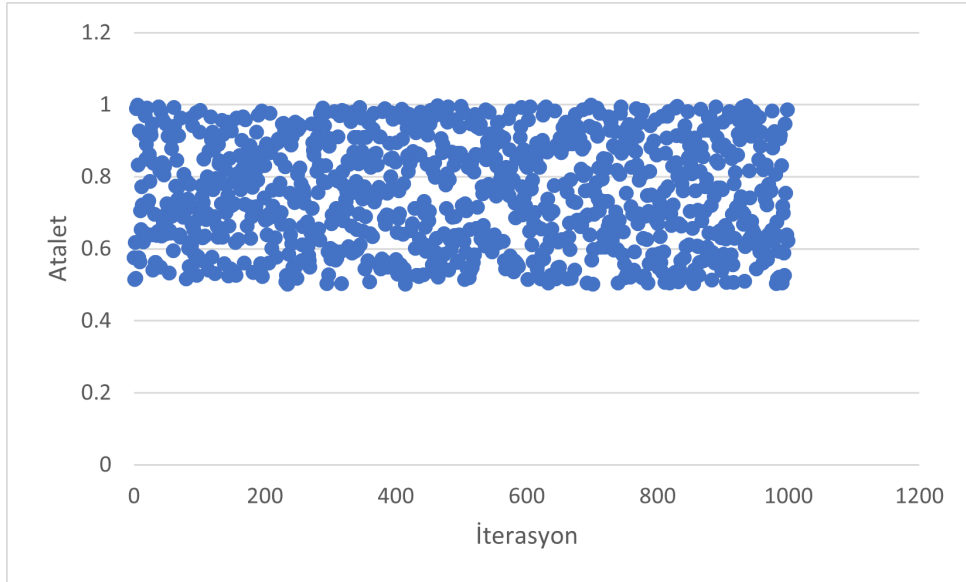


Şekil 3.5: Doğrusal Azalan Atalet Stratejisi - 2

Doğrusal Azalan Atalet Stratejisi, atalet değerini başlangıçta yüksek bir değerden (0.9) başlatarak, her iterasyonda azaltmaktadır. Bu strateji, PSO algoritmasının arama uzayını başlangıçta geniş bir şekilde aramasını sağlarken, daha sonraki iterasyonlarda daha küçük ve daha odaklı bir aramaya izin vermektedir. Sonuçlara göre, bu strateji en iyi uygunluk değerine $f(G_{Best}) = -0.980366$, 486 iterasyon sonunda ulaşmıştır



Şekil 3.6: Rastgele Atalet Stratejisi - 1



Şekil 3.7: Rastgele Atalet Stratejisi - 2

Rastgele Atalet Stratejisi, atalet değerini her iterasyonda rastgele bir şekilde belirlemektedir. Bu strateji, arama uzayının rastgele bir şekilde araştırılmasına izin vermektedir ve yerel optimumlardan kaçınmayı amaçlamaktadır. Sonuçlara göre, bu strateji en iyi uygunluk değerine $f(G_{Best}) = -0.0351$, 789 iterasyon sonunda ulaşmıştır

Sonuç

Rastgele Atalet Ağırlığı Stratejisi, atalet ağırlığını her iterasyonda kaotik bir şekilde değiştirerek arama sürecini daha dinamik ve öngörülemez hale getirir. Bu durum, algoritmanın daha geniş bir arama uzayını keşfetmesine ve yerel optimumlardan kaçınmasına olanak tanır. Kaotik Atalet Ağırlığı Stratejisi ise, atalet ağırlığını her iterasyonda rastgele bir değere ayarlayarak arama sürecine rastgelelik katar ve bu da yerel optimumlardan kaçınmayı sağlar.

Ancak, mevcut çalışmamızda Rastgele Atalet Ağırlığı ve Kaotik Atalet Ağırlığı, Bansal ve arkadaşlarının [3] çalışmasındaki kadar yüksek doğruluk ve verimlilik değerlerine ulaşamamıştır. Bu farklılık, kullanılan veri kümeleri, problem türleri ve diğer parametre ayarlarındaki farklılıklardan kaynaklanıyor olabilir. Özellikle, bizim çalışmamızda kullanılan veri kümesi veya problem, Rastgele Atalet Ağırlığı ve Kaotik Atalet Ağırlığı stratejilerinin avantajlarını tam olarak ortaya çıkarmayacak nitelikte olabilir.

Diğer taraftan, Doğrusal Azalan Ağırlık stratejisi, daha deterministik bir yaklaşım sergilemesine rağmen, bizim çalışmamızda diğer stratejilere benzer başarı oranları elde etmiştir. Bu durum, doğrusal azalan ağırlık stratejilerin, belirli problemler veya veri kümeleri için Rastgele Atalet Ağırlığı ve Kaotik Atalet Ağırlığı stratejilerine göre daha uygun olabileceğini göstermektedir.

Sonuç olarak, bu çalışmanın bulguları, Bansal ve arkadaşlarının çalışmasıyla kısmen uyumlu olup, bazı farklılıklar göstermektedir. Rastgele Atalet Ağırlığı ve Kaotik Atalet Ağırlığı stratejilerinin, yerel optimumlardan kaçınma ve geniş bir arama uzayını keşfetme potansiyeline sahip olduğu doğrulanırken, bu stratejilerin her zaman en iyi performansı göstermeyebileceği de görülmüştür. Bu nedenle, PSO algoritmasının performansını optimize etmek için, farklı atalet ağırlığı stratejilerinin, problem türüne ve veri kümesine göre dikkatlice seçilmesi ve parametrelerinin ayarlanması gerekmektedir.

3.2 Problem-2

Bu çalışmada, EU-DEM [13] veri setini kullanarak, Parçacık Sürü Optimizasyonu (PSO) algoritması ile belirtilen iki koordinat arasındaki alanda en düşük rakıma sahip noktayı bulacağız.

EU-DEM (European Digital Elevation Model), Avrupa'yı 25 metre çözünürlüğünde kapsayan bir veri setidir.

Hesaplamalar, Kartezyen koordinat sisteminde gerçekleştirilecektir. Bu nedenle, Coğrafi koordinat sisteminde (Derece:Dakika-ondalık) bulunan koordinatları Kartezyen koordinatlara dönüştürmemiz gerekmektedir. Bu dönüşüm için Eşdikdörtgensel izdüşüm (Equirectangular projection) [14] yöntemini kullanacağız. İlk koordinatı orijin olarak kabul edip, diğer koordinatları bu orijine göre dönüştüreceğiz.

Eşdikdörtgensel izdüşüm, hesaplamaları basitleştirmek amacıyla sıkça kullanılan bir yöntemdir. Ancak, özellikle geniş alanlarda veya ekvatorдан uzaklaştıkça doğruluğu azalır. Bu projeksiyon, Dünya'nın yüzeyini düz bir yüzey olarak varsaydığından, kutuplara yakın bölgelerde veya büyük enlem farkları olan bölgelerde bozulmalara yol açabilir. Bu nedenle, dönüşüm sırasında hassasiyet kaybı yaşanabilir.

Eşdikdörtgensel izdüşüm yöntemine göre, Coğrafi koordinatları ($lat0$, $lon0$) ve ($lat1$, $lon1$) için dönüştürme formülleri aşağıdaki gibidir:

$$x = R \cdot (lon1 - lon0) \cdot \cos\left(\frac{lat0 + lat1}{2}\right)$$

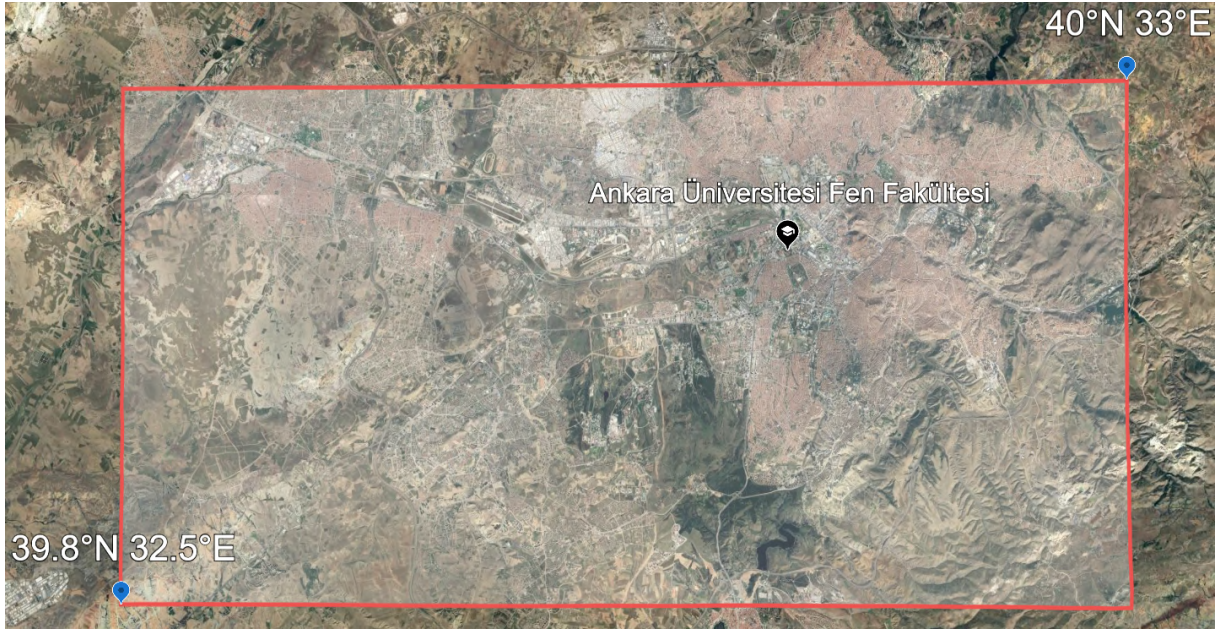
$$y = R \cdot (lat1 - lat0)$$

Burada R Dünya'nın yarıçapıdır (yaklaşık 6371 km). Bu formüller ile GPS koordinatlarını Kartezyen düzleme aktararak, PSO algoritmasında kullanılabilir hale getireceğiz.

PSO algoritmasının parametreleri şu şekildedir:

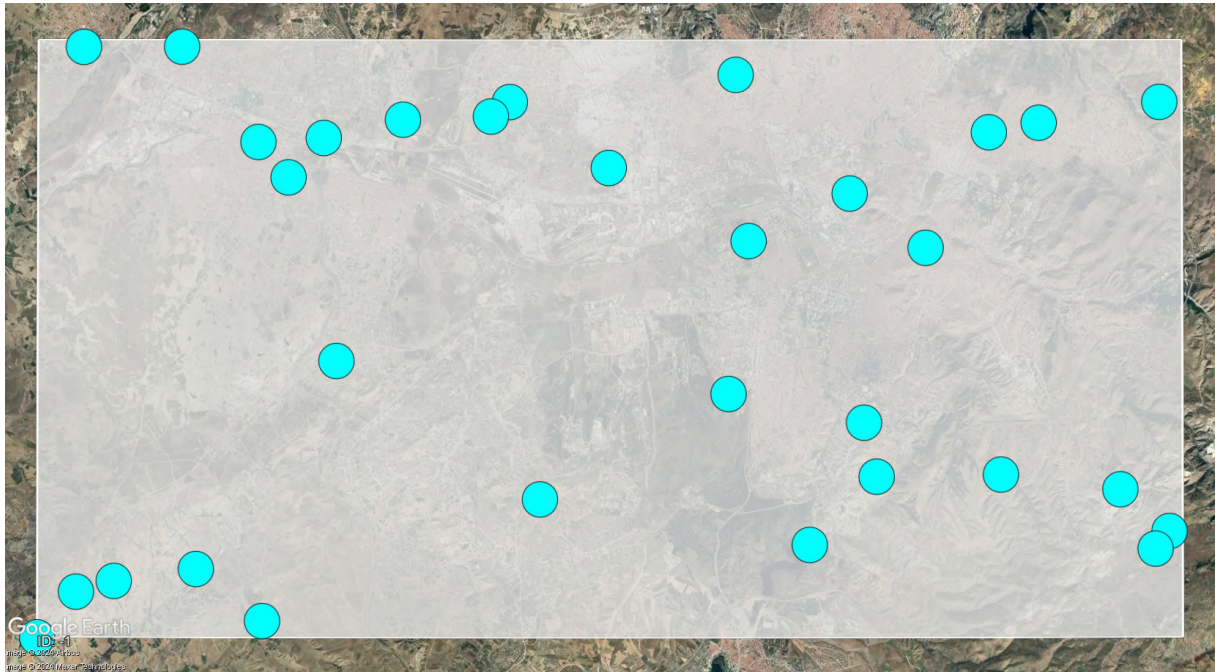
- Atalet katsayısı (inertia): 0.7
- Bireysel öğrenme katsayısı (cognitive): 0.5
- Sosyal öğrenme katsayısı (social): 0.5

39.8°N 32.5°E koordinatlarını orijin olarak seçelim ve 40°N 33°E koordinatları ile arasındaki alanda sürümüzü sınırlandıralım.



Şekil 3.8: Çalışma Alanı (Görsel Google Earth ile oluşturulmuştur)

PSO algoritması için rastgele konumlarda ve hızlarda parçacıkları oluşturalım:



Şekil 3.9: Parçacıkların Başlangıçtaki Konumları



Şekil 3.10: 1. İterasyon



Şekil 3.11: 2. İterasyon



Şekil 3.12: 11. İterasyon



Şekil 3.13: 21. İterasyon



Şekil 3.14: 46. İterasyon

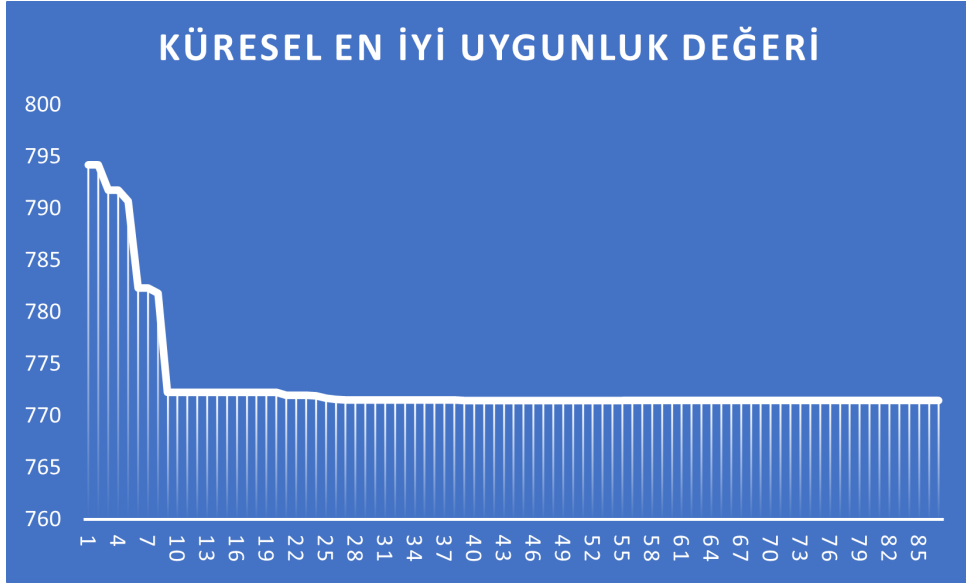


Şekil 3.15: 87. Son İterasyon

Görsellerde PSO algoritmasının iterasyonlar boyunca ilerleyişi gösterilmektedir. İlk iterasyonlarda (3.10) parçacıklar daha geniş bir alana dağılmışken, zamanla optimal çözüme yaklaştıkça belirli bir noktada kümelenmeye başlamışlardır (3.12). Bu durum, parçacıkların kendi en iyi deneyimleri ve sürünün en iyi deneyimi doğrultusunda hareket ederek çözüm uzayını daralttığını göstermektedir. Son iterasyonda (3.15) ise parçacıkların neredeyse tamamı aynı noktada toplanarak optimizasyon probleminin çözümüne ulaştığını ifade etmektedir.

Sonuç ve Çıkarımlar

Algoritmanın yakınsama grafiği aşağıdaki gibidir:



Şekil 3.16: PSO Algoritması Yakınsama Grafiği

Bu grafik, algoritmanın iterasyonlar boyunca uygunluk değerini nasıl iyileştirdiğini göstermektedir. Grafikte görüldüğü gibi, algoritma başlangıçta yüksek uygunluk değerleriyle başlamış, ancak iterasyonlar ilerledikçe uygunluk değerini düşürerek en iyi küresel çözüme yakınsamıştır.

PSO algoritması, 87 iterasyon boyunca çalıştırılmış, 55. iterasyon sonunda en iyi küresel çözümü bulmuştur. Algoritmanın yakınsama süreci aşağıdaki gibi özetlenebilir:

- İlk iterasyonlarda, algoritma geniş bir alanı taramış ve uygun noktaları keşfetmeye başlamıştır.
- 10. iterasyondan itibaren, en iyi küresel konum (39.94, 32.5) bulunmuş ve bu konumun etrafında yoğunlaşmaya başlamıştır.
- 25. iterasyonda, daha düşük bir uygunluk değerine (771.6678) sahip bir nokta (39.9461, 32.5006) bulunmuştur.
- 46. iterasyona kadar, algoritma küçük iyileştirmeler yaparak en iyi küresel konumu (39.9458, 32.5008) bulmuş ve uygunluk değerini 771.48486328125'e düşürmüştür.
- Son olarak, 55. iterasyonda en iyi küresel konum (39.9458, 32.5008) bulunmuş ve uygunluk değeri 771.4832'e düşürülmüştür.

Bu sonuçlara göre, belirtilen alan içerisinde en düşük rakıma sahip nokta (39.9458, 32.5008) koordinatlarında bulunmaktadır ve bu noktanın uygunluk değeri (yüksekliği) 771.4832 metredir.

PSO Algoritmasının Hızı

PSO algoritması, sürü zekası temelli bir optimizasyon algoritması olduğundan, genel olarak hızlı bir şekilde yakınsamaktadır. Bu çalışmada da görüldüğü gibi, algoritma 55 iterasyonda en iyi çözüme ulaşmıştır. Bu hız, algoritmanın basit yapısı ve az sayıda parametreye sahip olmasından kaynaklanmaktadır.

Ancak, algoritmanın hızı çeşitli faktörlere bağlıdır. Örneğin, parçacık sayısı, arama uzayının boyutu, uygunluk fonksiyonunun karmaşıklığı ve parametrelerin değerleri algoritmanın hızını etkileyebilir. Genel olarak, daha fazla parçacık ve daha karmaşık problemler için algoritmanın yakınsama süresi uzayacaktır.

3.3 Problem-3

Bu örnekte, bir grup turistin bir şehirdeki en iyi oteli bulma problemi ele alınmıştır. Turistlerin hedefi, şehirdeki oteller arasında en yüksek puana sahip olanı keşfetmektir. Optimizasyon sürecinde, otellerin puanları 1 ile 10 arasında değişmektedir.

Parçacıkların Tanımı

- **Parçacık:** Her bir turist bir parçacık olarak modellenmiştir.
- **Pozisyon:** Her turistin mevcut otel seçimi, yani otelin puanı.
- **Hız:** Turistin bir sonraki otele geçiş hızı ve yönü.

İlk Pozisyon ve Hızların Belirlenmesi

Turistler rastgele otellerde başlangıç yapar. Başlangıç pozisyonları turistlerin bulundukları otellerin puanlarıdır. Örneğin, beş turistin başlangıç pozisyonları $[5, 7, 6, 4, 8]$ şeklinde olabilir. Hızlar ise rastgele belirlenir, örneğin, $[-0.5, 0.3, -0.2, 0.1, -0.4]$.

En İyi Bireysel ve Küresel Değerlerin Belirlenmesi

Her turistin şimdiye kadar gördüğü en yüksek puanlı otel, bireysel en iyi ($pBest$) olarak adlandırılır. Tüm turistlerin gördüğü en yüksek puanlı otel ise küresel en iyi ($gBest$) olarak tanımlanır. Başlangıçta, $pBest$ her turistin mevcut otelinin puanı, $gBest$ ise tüm turistlerin otelleri arasında en yüksek puana sahip olanıdır.

İterasyonlar

Turistler belirli sayıda iterasyon boyunca otelleri ziyaret eder ve her iterasyonda hız ve pozisyonlarını güncellerler. Her iterasyonda:

1. Turistlerin yeni otelleri ziyaret etmeleri (pozisyon güncelleme).
2. Otellerin puanlarını kontrol etmeleri.
3. Kendi en iyi ($pBest$) ve tüm turistlerin en iyi ($gBest$) puanlarını güncellemeleri.

Örnek İterasyon:

- 1. İterasyon:
 - Turist 1 (Başlangıç: 5): Yeni pozisyon $5 + (-0.5) = 4.5$ (Otel 4.5 puan)
 - Turist 2 (Başlangıç: 7): Yeni pozisyon $7 + 0.3 = 7.3$ (Otel 7.3 puan)
 - Diğer turistlerin güncellemeleri de benzer şekilde hesaplanır.
- 2. İterasyon:
 - Güncellemeler sonrası $pBest$ ve $gBest$ tekrar hesaplanır.

Bu adımlar belirli bir iterasyon sayısı veya belirli bir kriter sağlanana kadar devam eder. Sonunda $gBest$ değeri, yani tüm turistlerin bulduğu en iyi otel puanı, şehirdeki en iyi oteli gösterir.

Bölüm 4

Sonuç

Bu çalışmada, Parçacık Sürü Optimizasyonu (PSO) algoritmasının temel prensipleri ve çeşitli problemlere uygulanması incelenmiştir. Çalışma kapsamında, PSO algoritmasının farklı uygulama alanları ve optimum çözümleri bulma yeteneği vurgulanmıştır. Uygulama örneklerinden elde edilen sonuçlara göre, PSO algoritması Drop-Wave, yükseklik verisi (rakım) ve otel puanlama gibi çeşitli optimizasyon problemlerinde başarılı bir şekilde uygulanmıştır.

Performans analizi bağlamında, farklı atalet ağırlığı stratejilerinin PSO algoritmasının yakınsama davranışını nasıl etkilediği incelenmiştir. Kaotik, doğrusal azalan ve rastgele atalet ağırlığı stratejileri denenmiş ve sonuçlar Bansal ve arkadaşlarının araştırmasıyla karşılaştırılmıştır. Bu karşılaştırma sonucunda, her optimizasyon problemi için ortak bir atalet ağırlığı stratejisinin uygun olmadığı ortaya konmuştur.

Ayrıca, PSO'nun optimizasyon performansının atalet ağırlığı stratejisine göre önemli ölçüde değişebildiği gösterilmiştir. Bunun yanı sıra, bu çalışma PSO'nun gerçek dünya optimizasyon problemlerine başarıyla uygulanabileceğini göstermiştir. Dinamik arama stratejileri ve sürü davranışı sayesinde, PSO algoritması geniş bir arama uzayında en uygun çözümlere yakınsamayı başarmıştır. Ancak, daha iyi sonuçlar elde etmek için algoritmanın geliştirilmesi ve iyileştirilmesine yönelik çalışmalar yapılmalıdır. PSO'nun esnekliği ve uyarlanabilirliği, bu alandaki gelecekteki çalışmalar için umut verici bir temel sunmaktadır.

Kaynakça

- [1] Wali Mashwani, Ruqayya Ruqayya, and Samir Brahim Belhaouari. A multi-swarm intelligence algorithm for expensive bound constrained optimization problems. *Complexity*, 2021:1–18, 02 2021.
- [2] J. Kennedy and R. Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95 - International Conference on Neural Networks*, volume 4, pages 1942–1948 vol.4, 1995.
- [3] J. C. Bansal, P. K. Singh, Mukesh Saraswat, Abhishek Verma, Shimpi Singh Jadon, and Ajith Abraham. Inertia weight strategies in particle swarm optimization. In *2011 Third World Congress on Nature and Biologically Inspired Computing*, pages 633–640, 2011.
- [4] M. R. AlRashidi, M. F. AlHajri, A. K. Al-Othman, and K. M. El-Naggar. *Particle Swarm Optimization and Its Applications in Power Systems*, pages 295–324. Springer Berlin Heidelberg, Berlin, Heidelberg, 2010.
- [5] Abdullah Yılmaz. A machine learning-based framework using the particle swarm optimization algorithm for credit card fraud detection. *Communications Faculty of Sciences University of Ankara Series A2-A3 Physical Sciences and Engineering*, 66:82–94, 01 2024.
- [6] Yaya Sylla, Adama Coulibaly, and Pierre Morizet. Towards deep learning: A comprehensive overview on pso with machine learning. Avestia Publishing, 2022.
- [7] Tiago Sousa, Arlindo Silva, and Ana Neves. Particle swarm based data mining algorithms for classification tasks. *Parallel Computing*, 30(5):767–783, 2004. Parallel and nature-inspired computational paradigms and applications.
- [8] Axel Thevenot. Particle swarm optimization visually explained, Feb 2022.
- [9] Mihrimah Özmen, Yılmaz Delice, and Emel Kızılkaya Aydoğan. Telekomünikasyon sektöründe pso ile müşteri bölümlemesi. *Bilişim Teknolojileri Dergisi*, 11(2):163–173, 2018.
- [10] Y. Shi and R. Eberhart. A modified particle swarm optimizer. In *1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence (Cat. No.98TH8360)*, pages 69–73, 1998.
- [11] R.C. Eberhart and Yuhui Shi. Tracking and optimizing dynamic systems with particle swarms. In *Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat. No.01TH8546)*, volume 1, pages 94–100 vol. 1, 2001.

- [12] UNIT ANIL KILIC. Particle swarm optimization library. <https://github.com/unitanilkilic/pso>, 2024. MIT License, v1.0.
- [13] European Environment Agency. [deprecated] digital elevation model over europe (eu-dem). <http://data.europa.eu/88u/dataset/eu-dem>, Accessed 2024. Accessed: 2024-05-24.
- [14] Wikipedia contributors. Equirectangular projection — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Equirectangular_projection&oldid=1185683074, 2023. [Online; accessed 27-May-2024].